

# Laboratorio di Calcolo

## A.A. 2026/2027, Canale A-Dec

**Giacomo Artoni**, Lorenzo Caprini

Metodo di lavoro del programmatore

# Che cos'è un algoritmo?

Un **algoritmo** è una sequenza *finita* di istruzioni *non ambigue* che, a partire da dati in ingresso, produce un risultato in un tempo finito.

- **Finitezza:** termina dopo un numero finito di passi.
- **Non ambiguità:** ogni passo ha un solo significato possibile.
- **Eseguibilità:** ogni passo è un'operazione elementare che l'esecutore sa compiere.
- **Ingresso e uscita:** riceve dei dati e restituisce un risultato.
- Un algoritmo **non dipende dal linguaggio**: lo stesso algoritmo può essere scritto in italiano, in pseudocodice, in C o in Python.

Esempio: MCD di Euclide

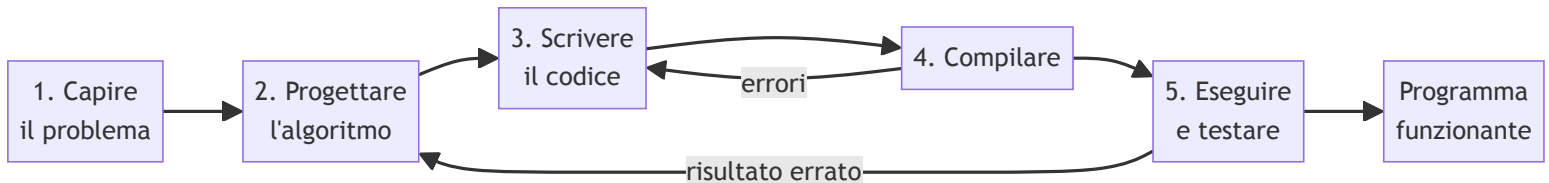
Dati due interi positivi $a$ e $b$:

1. Se $b = 0$, il risultato è $a$. **Fine.**
2. Calcola il resto $r$ della divisione $a/b$.
3. Poni $a \leftarrow b$ e $b \leftarrow r$.
4. Torna al passo 1.

$\mathrm{MCD}(48, 18)$:
$(48, 18) \to (18, 12) \to (12, 6) \to (6, 0) \Rightarrow 6$

# Dal problema al programma

Scrivere codice è solo **una** delle fasi del lavoro. Il ciclo completo:




- Le fasi **1** e **2** si fanno con **carta e penna**, *prima* di toccare la tastiera.
- Le fasi **3–5** si ripetono molte volte: è normale, non è un fallimento!

# Un esempio scientifico: la radice quadrata

**Problema:** calcolare $\sqrt{a}$ usando solo le quattro operazioni di base.

## 1. Capire il problema

- **Dati:** un numero reale $a > 0$ e una tolleranza $\varepsilon$.
- **Risultato:** un $x$ tale che $x^2 \approx a$.

## 2. Progettare l'algoritmo (metodo di Erone)

Se $x$ è una stima di $\sqrt{a}$, allora la media tra $x$ e $a/x$ è una stima migliore:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

```
// pseudocodice
leggi a, ε
x ← a
ripeti:
  x_nuovo ← (x + a/x) / 2
  se |x_nuovo - x| < ε: esci
  x ← x_nuovo
stampa x_nuovo
```

Con $a = 2$:

$x_0 = 2 \to 1.5 \to 1.416667 \to 1.414216 \to 1.414214$

Bastano 4 iterazioni per 7 cifre corrette!

# Il terminale: comandi essenziali

Lavoreremo quasi sempre dalla **riga di comando** (vedi la prima esercitazione).

| Comando | Significato |
|---|---|
| `pwd` | Mostra la cartella corrente |
| `ls` / `ls -l` | Elenca i file (con dettagli) |
| `cd nome` | Entra nella cartella `nome` |
| `cd ..` | Risale di un livello |
| `mkdir nome` | Crea una cartella |
| `cp a b` | Copia il file `a` in `b` |
| `mv a b` | Sposta o rinomina `a` in `b` |
| `rm file` | Cancella un file (**senza cestino!**) |

## Trucchi che fanno risparmiare tempo

- **Tab:** completa automaticamente nomi di file e comandi.
- **Freccia ↑:** richiama i comandi precedenti.
- **Ctrl+C:** interrompe un programma bloccato.
- `clear` : ripulisce lo schermo.

**Attenzione:** `rm` cancella definitivamente.
Non esiste "annulla".

# Il ciclo modifica → compila → esegui

```
$ mkdir -p ~/labcalc/modulo07 && cd ~/labcalc/modulo07   # cartella di lavoro
$ gedit erone.c &                                         # editor (o VS Code, nano, emacs ... )
$ gcc -Wall -Wextra -o erone erone.c                      # compilazione
$ ./erone                                                 # esecuzione
```

## Le opzioni che useremo **sempre**

- `-Wall -Wextra` : chiedono al compilatore di segnalare **tutti** i costrutti sospetti (*warning*).
- `-o erone` : nome dell'eseguibile. Senza questa opzione `gcc` produce un file chiamato `a.out` .

**Regola d'oro:** procedete a **piccoli passi**. Scrivete poche righe, compilate, eseguite, controllate. Cento righe scritte di fila producono cento errori difficili da trovare.

# Tipi di errore

Non tutti gli errori si manifestano nello stesso momento.

| Tipo | Quando si manifesta | Chi lo trova | Esempio |
|---|---|---|---|
| **Sintassi** | Compilazione | Compilatore | `;` mancante |
| **Warning** | Compilazione | Compilatore | Variabile non inizializzata |
| **Linking** | Compilazione (fase finale) | Linker | `printff` invece di `printf` |
| **Runtime** | Esecuzione | Sistema operativo | Divisione intera per zero |
| **Logico** | Esecuzione | **Solo voi!** | Formula sbagliata |

Andando verso il basso nella tabella gli errori diventano **più subdoli**: un errore logico non produce alcun messaggio, solo un risultato sbagliato.

# Leggere un messaggio di errore

```c
#include <stdio.h>

int main() {
    double raggio = 2.0;
    double area;
    area = 3.14159 * ragio * raggio;
    printf("Area: %f\n", area);
    return 0;
}
```

```
$ gcc -Wall -Wextra -o area area.c
area.c: In function ‘main’:
area.c:6:22: error: ‘ragio’ undeclared (first
  use in this function); did you mean ‘raggio’?
    6 |     area = 3.14159 * ragio * raggio;
      |                      ^~~~~
      |                      raggio
```

- `area.c:6:22` : **file**, **riga** e **colonna**.
- `error:` la compilazione **fallisce**, nessun eseguibile.
- Il compilatore spesso **suggerisce** la correzione.

Un singolo errore può generarne molti altri a cascata: correggete **il primo**, ricompilate, ripetete.

# I warning: il compilatore vi avvisa

```c
#include <stdio.h>

int main() {
    int n;
    int doppio = 2 * n;
    int triplo = 3 * n;

    printf("%d\n", doppio);
    return 0;
}
```

```
$ gcc -Wall -Wextra -o warn warn.c
warn.c: In function ‘main’:
warn.c:6:9: warning: unused variable ‘triplo’
  [-Wunused-variable]
warn.c:5:9: warning: ‘n’ is used uninitialized
  [-Wuninitialized]
```

- Con i warning il programma **compila** e l'eseguibile viene creato…
- …ma quasi sempre c'è un **bug**: qui `n` contiene un valore casuale!

Trattate i warning **come errori**: un programma consegnato deve compilare senza alcun messaggio.

# Errori di linking

Il compilatore traduce il vostro file, ma è il **linker** a collegarlo alle funzioni di libreria (come `printf`).

```c
#include <stdio.h>

int main() {
    printff("Ciao!\n");
    return 0;
}
```

```
$ gcc -Wall -Wextra -o ciao ciao.c
ciao.c: In function ‘main’:
ciao.c:4:5: warning: implicit declaration of function ‘printff’; did you mean ‘printf’?
/usr/bin/ld: /tmp/ccQ2Lm9a.o: in function `main':
ciao.c:(.text+0×14): undefined reference to `printff'
collect2: error: ld returned 1 exit status
```

- `undefined reference to ...`: il linker non trova la funzione da nessuna parte.
- Cause tipiche: nome scritto male, `main` scritto `Main`, libreria non collegata (es. `-lm`, lo vedremo più avanti).

# Errori a runtime ed errori logici

Il programma compila senza messaggi… ma non funziona.

## Errore a runtime

```
1   int n = 0;
2   int x = 10 / n;
```

```
1   $ ./div
2   Floating point exception (core dumped)
```

Il programma viene **interrotto** dal sistema operativo.

## Errore logico

```
1   double a = 4.0, b = 6.0;
2   double media = a + b / 2;
3   printf("Media: %f\n", media);
```

```
1   $ ./media
2   Media: 7.000000
```

Nessun messaggio, risultato ovviamente **sbagliato** (riuscite a trovare l'errore?).

Per riuscire a trovare gli errori logici è importante avere un'idea di **quale risultato aspettarsi**.

# Testare un programma

Un programma che "gira" non è necessariamente un programma **corretto**.

- **Casi noti:** scegliete valori di input per i quali conoscete già il risultato.
- **Casi limite:** zero, numeri negativi, numeri molto grandi o molto piccoli.
- **Proprietà attese:** simmetrie, proporzionalità, leggi di scala.
- **Controllo fisico:** ordini di grandezza e unità di misura sensati.
- **Un cambiamento alla volta:** dopo ogni modifica, ripetete i test.

Esempio: area del cerchio $A = \pi r^2$

| Ingresso | Atteso | Cosa verifica |
|---|---|---|
| $r = 1$ | $3.14159\ldots$ | Caso noto |
| $r = 0$ | $0$ | Caso limite |
| $r = 2$ | $4 \times A(1)$ | Legge di scala |

Per il metodo di Erone: $\sqrt{4} = 2$, $\sqrt{1} = 1$, e controllate che $x^2$ sia vicino ad $a$.

# Strategie di debug

Quando il risultato è sbagliato, procedete con **metodo**, non a tentativi.

1. **Riproducete** l'errore con l'ingresso più semplice possibile.
2. **Stampate i valori intermedi** con `printf`: dove il valore smette di essere quello atteso?
3. **Isolate** il problema: commentate parti di codice e riducete il programma all'osso.
4. **Rileggete il codice ad alta voce**, spiegandolo riga per riga (per esempio a chi vi sta a fianco in laboratorio!).
5. **Cambiate una cosa alla volta**, ricompilate e ripetete i test.

Il debug è una parte **normale** del lavoro: anche i programmatori esperti passano più tempo a correggere che a scrivere codice.